

Stockholms Universitet, Matematiska institutionen

Programmeringsteknik, DA2005 ST26

Sally Altaie

Datum: 14/08-2026

Rapport

1. Inledning	2
2. Installation och användning	2
3. Bibliotek	2
4. Programstruktur	3
4.2 Design	3
4.3 Testning	4
5. Reflektion	4

1. Inledning

Jag har implementerat uppgift 2 i projektet ASCII Art Studio. Projektet är ett kommandobaserat program utvecklat i Python som omvandlar bildfiler till textrepresentation med ASCII-tecken.

Programmet kan hantera flera bilder i samma session och gör det möjligt att ladda bilder, använda alias, justera bildinställningar, rendera bilder till ASCII-konst samt spara resultatet som textfiler. Programmet är utvecklat med objektorienterad programmering och är uppdelat i moduler med tydliga ansvarsområden.

2. Installation och användning

Programmet kräver Python 3.13.7 och Pillow 12.3.0.

Installera projektets beroenden via terminalen med kommandot:

- `pip install -r requirements.txt`

För att sedan starta programmet:

- `python main.py`

Vid programstart ska en kommandotolk visas där användaren kan skriva kommandon för att ladda bilder, ändra inställningar, rendera ASCII-konst samt spara och läsa in sessioner.

Notera att en bild måste placeras i projektets bildmapp, *images*. Programmet har testats med bildformaten JPG och PNG.

Exempel på en körning:

- `load image images/stadshuset.jpg`
- `set current width 80`
- `set current brightness 1.5`
- `render`
- `render current to output/stadshuset.txt`
- `save session as sessions/min_session.json`
- `quit`

3. Bibliotek

Projektet använder biblioteket Pillow för att läsa in och bearbeta bildfiler innan de konverteras till ASCII-text. Pillow är det enda externa biblioteket som används i projektet och installeras via terminalen genom kommandot:

- `pip install -r requirements.txt`

För att läsa in eller spara sessioner används Python-modulen json och för projektets tester används unittest. Dessa moduler ingår i Pythons standardbibliotek och kräver därför ingen specifik installation.

4. Programstruktur

Programmet är uppdelat i flera filer och mappar där varje del har ett tydligt ansvarsområde. Bilder som ska bearbetas placeras i mappen images, genererade ASCII-filer sparas i output, sparade sessioner lagras i sessions och projektets enhetstester finns i tests.

Mappstruktur

images/ - innehåller bildfiler som används av programmet.

output/ - innehåller genererade ASCII-textfiler.

sessions/ - innehåller sparade sessioner i JSON-format.

tests/ - innehåller enhetstester för projektet.

tests/test_data/ - innehåller testdata som används av enhetstesterna

Filstruktur

ascii_image.py - innehåller klassen ArtImage som representerar en bild och dess inställningar.

command_parser.py - innehåller klassen CommandParser som tolkar användarens kommandon.

session.py - innehåller klassen Session som hanterar bilder och sparade sessioner.

renderer.py - innehåller funktioner för bildbearbetning och konvertering till ASCII-konst.

main.py - startar programmet och hanterar kommandoloopen.

README.md - innehåller en beskrivning av projektet, installation, användning och testning.

requirements.txt - innehåller projektets externa beroenden.

4.2 Design

Under utveckling av programmet har fokuset legat på att dela upp funktionaliteten i mindre moduler med tydliga ansvarsområden.

Renderingen har placerats i en separat modul, *renderer.py*, eftersom modulen ansvarar för bildbearbetning och konvertering till ASCII-konst. Detta gör att renderingslogiken kan återanvändas utan att vara beroende av de övriga moduler i programmet.

Programmets huvudsakliga datastruktur är en dictionary som används i klassen Session för att lagra alla inlästa bilder. Nyckeln utgörs av bildens filnamn eller alias och värdet av ett ArtImage objekt. Valet av dictionary gjordes för att den ger snabb åtkomst till bilderna. Listor används vid skapandet av ASCII-tecken där varje rad lagras innan de sammanfogas till en färdig textfil.

Under renderingen bearbetas bilden stegvis. Först justeras bildens ljusstyrka och kontrast, därefter ändras storlek och bilden konverteras till gråskala. Slutligen översätts varje pixel till ett ASCII-tecken utifrån dess ljusstyrka. Anledningen till att det är bra att skala ned bildens storlek innan rendering är för att minska mängden data som behöver bearbetas. Det bidrar till en snabbare rendering och en lägre minnesanvändning.

Felhantering har placerats där fel kan uppstå, exempelvis filhantering, inläsning av JSON-filer och ändring av bildinställningar. Tydliga felmeddelanden har implementerats som hjälper användaren att förstå orsaken till felet och korrigera felaktig inmatning utan att programmet avslutas oväntat.

4.3 Testning

Enhetstester har utvecklats med hjälp av Python-modulen unittest för att verifiera att programmets funktionalitet fungerar som förväntat.

Testerna verifierar programmets centrala funktioner, såsom inläsning av bilder, ändring av bildinställningar, rendering, sparande av ASCII-filer samt sparande och inläsning av sessioner. Det omfattar samtliga klasser i projektet samt de centrala funktionerna i `renderer.py`. Enhetstesterna körs via kommandot:

- `python -m unittest discover -s tests`

5. Reflektion

Projektet har gett mig en bättre förståelse för hur större projekt kan struktureras med hjälp av objektorienterad programmering. Genom att dela programmet i olika moduler och klasser har projektet varit överskådligt och enklare att utveckla under processens gång. Det har även visat hur viktigt det är att skapa en struktur och en plan för hur ett projekt ska se ut redan från början för att undvika att trassla in sig i för invecklade funktioner, filer som blir för stora eller att det är svårt att hitta det man letar efter.

Genom att kontinuerligt arbeta med enhetstester och testa programmet under utvecklingen blev det tydligt hur viktigt det är att tidigt verifiera programmets funktionalitet och snabbt upptäcka buggar och fel.

Utmaningen i projektet har varit att skapa funktioner som är lätta att följa. Eftersom projektet växte i omfattning blev det viktigt att dela upp programmet i klasser med tillhörande metoder för att undvika långa och svåröverskådliga kodblock.